



Activity 2 — Build the To-Do App

Time: ~25 minutes · **File you edit:** `App.js` · **Goal:** add, complete, and delete tasks — and show how many are left.

You'll practice today's ideas: `useState`, `onPress`, `TextInput`, `spread/filter/map`, and `FlatList` + `props`.



Start it up

```
git checkout session-2
npm install
npx expo start
```



The `session-2` branch already has the `finished` app, so you can always peek. To practice, try building each piece yourself first, then compare.

Build it in 5 moves

1 Two pieces of state

```
const [text, setText] = useState('');           // what's typed
const [tasks, setTasks] = useState([           // the list
  { id: 1, title: 'Tap me to finish 🟢', done: false },
]);
```

2 A controlled input + add button

```
<TextInput
  placeholder="Add a task..."
  value={text}
  onChangeText={setText}
  onSubmitEditing={addTask}
/>
<TouchableOpacity onPress={addTask}>
  <Text>+</Text>
</TouchableOpacity>
```

3 Add a task (make a new array!)

```
function addTask() {
  if (text.trim() === '') return;           // no empty tasks
  const newTask = { id: Date.now(), title: text, done: false };
  setTasks([newTask, ...tasks]);           // spread keeps the old ones
  setText('');                             // clear the box
}
```

4 Toggle done & delete

```
function toggleTask(id) {
  setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
}
function deleteTask(id) {
  setTasks(tasks.filter((t) => t.id !== id));
}
```

5 Show the list with **FlatList**

```
<FlatList
  data={tasks}
  keyExtractor={({item}) => String(item.id)}
  ListEmptyComponent={<Text>No tasks yet ✨</Text>}
  renderItem={({ item }) => (
    <View style={styles.item}>
      <TouchableOpacity onPress={() => toggleTask(item.id)}>
        <Text>{item.done ? '✅' : '❑'} {item.title}</Text>
      </TouchableOpacity>
      <TouchableOpacity onPress={() => deleteTask(item.id)}>
        <Text>🗑️</Text>
      </TouchableOpacity>
    </View>
  )}
/>
```

And a **counter** above the list:

```
const remaining = tasks.filter((t) => !t.done).length;
// ...
<Text>{remaining} left to do</Text>
```

✅ Checkpoints — you're done when...

- ☐ Typing + **+** (or the keyboard "done") **adds** a task to the top
- ☐ The input **clears** after adding
- ☐ **Tapping** a task toggles **✅** / **❑**
- ☐ **🗑️** removes a task
- ☐ The **"left to do"** count is always correct
- ☐ An **empty** message shows when there are no tasks

🌟 Stretch goals

1. **Strike-through done tasks.** The style already exists — wire it up:

```
<Text style={[styles.itemText, item.done && styles.itemTextDone]}>
  {item.title}
</Text>
```

2. **"Clear done" button** — remove all completed tasks:

```
function clearDone() {
  setTasks(tasks.filter((t) => !t.done));
}
```

3. **Move done tasks to the bottom** (hint: sort a copy before rendering).

4. **Extract a `TodoItem` component** that receives `task`, `onToggle`, `onDelete` as **props**.

😞 Stuck? Quick fixes

Problem	Try this
Button fires immediately / loops forever	You wrote <code>onPress={addTask()}</code> — drop the <code>()</code> → <code>onPress={addTask}</code>
Can't type in the box	Add both <code>value={text}</code> and <code>onChangeText={setText}</code>
List won't update	Don't use <code>tasks.push(...)</code> ; build a new array with <code>[..., ...tasks]</code>
"Each child needs a unique key" warning	Add <code>keyExtractor={(item) => String(item.id)}</code>
Everything toggles at once	Make sure each task has its own unique <code>id</code>

🏠 Take-home

Add the **"Clear done"** button and the **strike-through** style (stretch goals 1 & 2).

Next session: close the app and your tasks vanish 😱 — we fix that *forever* with a real **SQLite** database and turn this into a finished app. 💾